

# Econ 238 Project 1

Genevieve DiBari, Vicky Dinov, Claire Petersen  
April 23

# Method Overviews

## Cutting-Plane (Kelley) Algorithm

- Instead of minimizing  $f(x)$  directly, we introduce  $\theta$  and minimize  $\theta$  subject to  $\theta \geq f(x)$
- Since  $f(x)$  is convex, the tangent at any point  $x_i$  is below the optimal value.

---

**Algorithm 1** Cutting-Plane (Kelley) Method

---

```
1: Initialize:  $x_0 \in \mathcal{X}$ ,  $UB = f(x_0)$ ,  $LB = -\infty$ ,  $\varepsilon = 10^{-4}$ 
2: for  $k = 1, 2, \dots, K_{\max}$  do
3:   Solve:  $(x_k^*, \theta_k^*) = \arg \min_{x \in \mathcal{X}, \theta} \theta$ 
4:   s.t.  $\theta \geq f(x_j) + g_j^\top (x - x_j) \quad \forall j = 0, \dots, k-1$ 
5:    $LB = \theta_k^*$ 
6:    $UB = \min(UB, f(x_k^*))$ 
7:   if  $UB - LB \leq \varepsilon$  then
8:     return  $x_k^*, f(x_k^*)$ 
9:   end if
10: end for
```

---

# Method Overviews

## Projected Subgradient Method

- Iteratively moves in the negative subgradient direction with a diminishing step size, alpha, projecting each iterate back onto the feasible set.
- Terminates when the step size becomes small enough that iterates have effectively stopped moving.

---

**Algorithm 2** Projected Subgradient Method

---

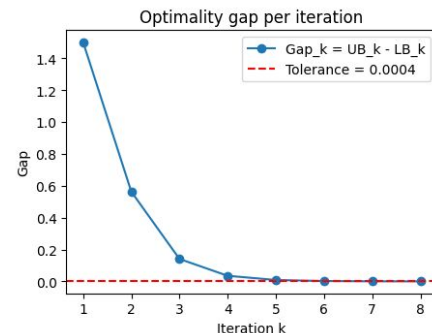
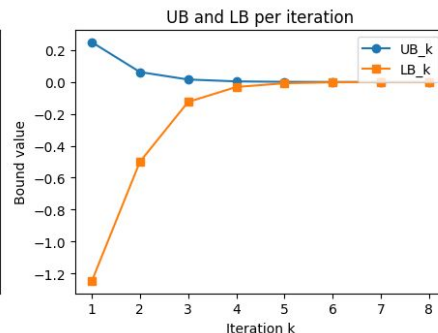
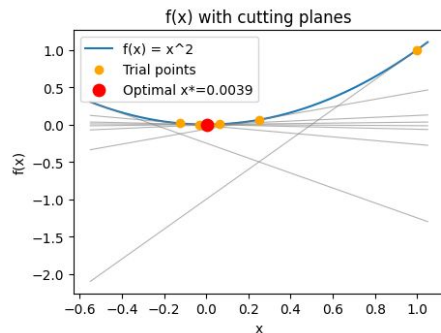
```
1: Initialize:  $x_0 \in \mathcal{X}$ ,  $x_{\text{best}} = x_0$ ,  $\varepsilon = 10^{-4}$ 
2: for  $k = 1, 2, \dots, K_{\text{max}}$  do
3:    $\alpha_k = \frac{1}{\sqrt{k}}$ 
4:    $x_k = \Pi_{\mathcal{X}}(x_{k-1} - \alpha_k g_{k-1})$    where  $g_{k-1} \in \partial f(x_{k-1})$ 
5:   if  $f(x_k) < f(x_{\text{best}})$  then
6:      $x_{\text{best}} = x_k$ 
7:   end if
8:   if  $\|x_k - x_{k-1}\| \leq \varepsilon$  then
9:     return  $x_{\text{best}}, f(x_{\text{best}})$ 
10:  end if
11: end for
```

---

$$P1 \cdot f(x) = x^2 \text{ on } [-0.5, 1]$$

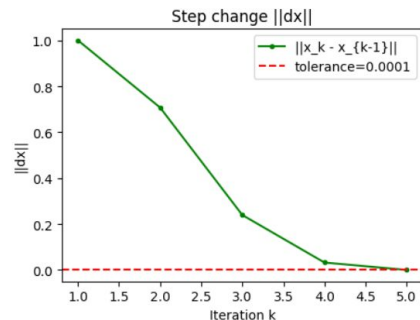
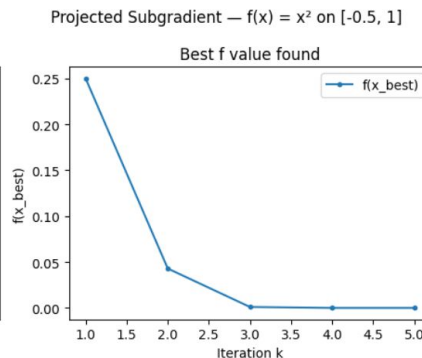
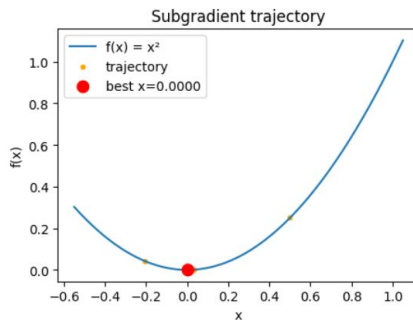
## Cutting-Plane

Converged at iteration: **8**  
 Optimal  $x \approx 0.003906$   
 Optimal  $f \approx 0.000015$   
 Final Gap: 0.000137  
 CPU Time = 0.091379



## Projected Subgradient

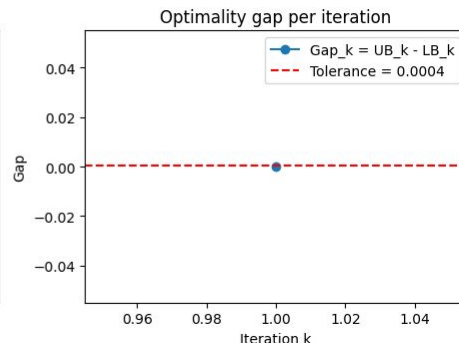
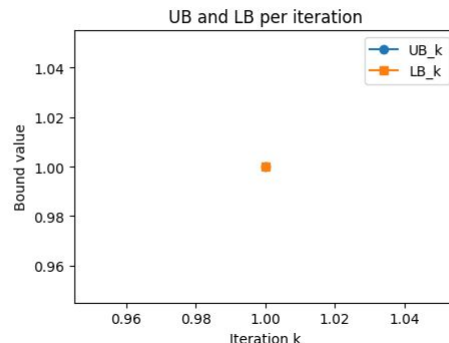
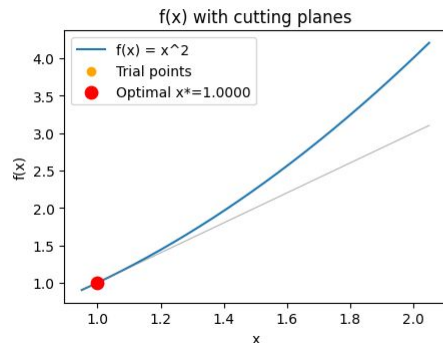
Converged at iteration: **5**  
 Optimal  $x \approx 0.000000$   
 Optimal  $f \approx 0.000000$   
 Gap = 0.000000  
 CPU = 0.000871



$$P2 \cdot f(x) = x^2 \text{ on } [1, 2]$$

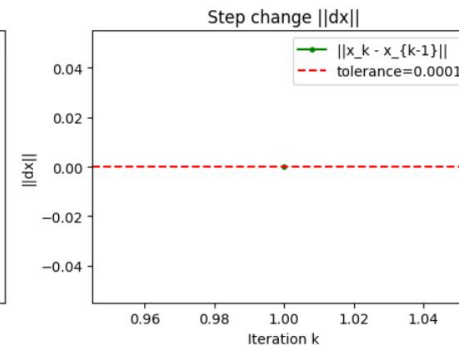
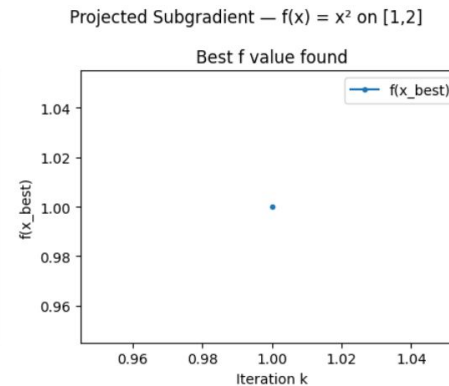
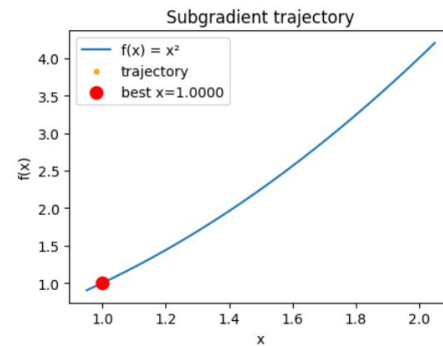
## Cutting-Plane

Converged at iteration: **1**  
 Optimal  $x \approx 1.000000$   
 Optimal  $f \approx 1.000000$   
 CPU = 0.01385617  
 Gap = 0.00000



## Projected Subgradient

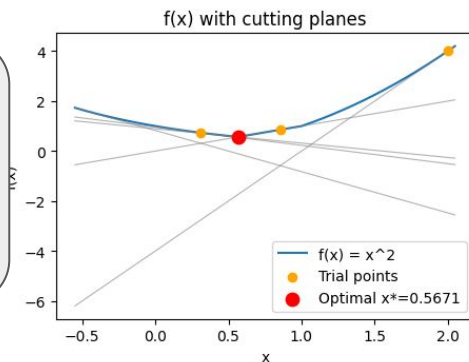
Converged at iteration: **1**  
 Optimal  $x \approx 1.000000$   
 Optimal  $f \approx 1.000000$   
 CPU = 0.01076  
 Gap = 0.000000



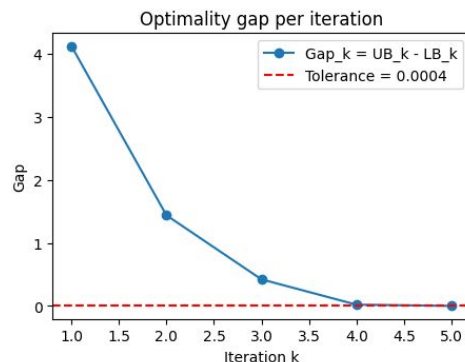
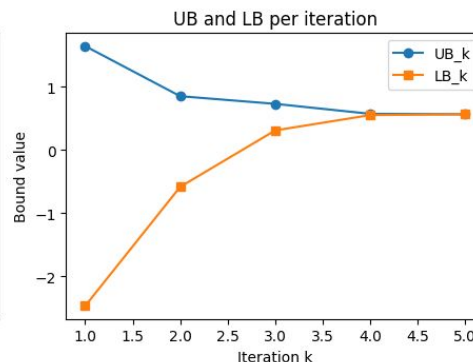
P3 ·  $f(x) = \max(e^{-x}, x, x^2)$  on  $[-0.5, 2]$

## Cutting-Plane

Converged at iteration: **5**  
 Optimal  $x \approx 0.567114$   
 Optimal  $f \approx 0.567160$   
 CPU = 0.2925370  
 Gap = .0000046

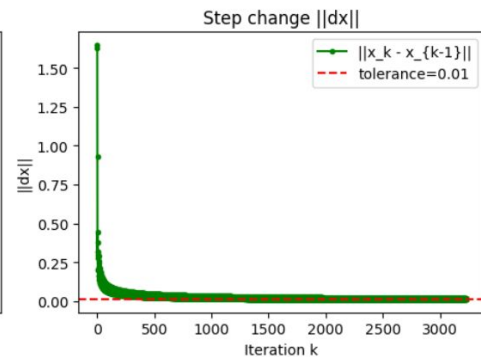
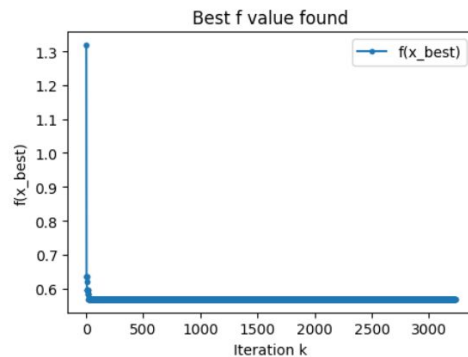
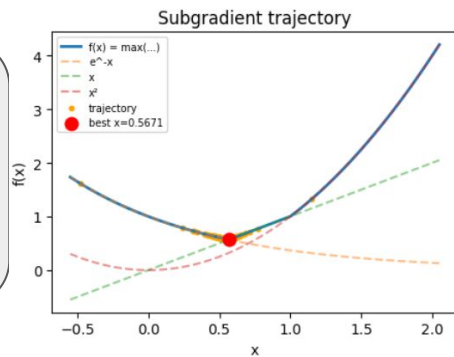


Cutting Plane Method —  $f(x) = x^2$  on  $[1, 2]$



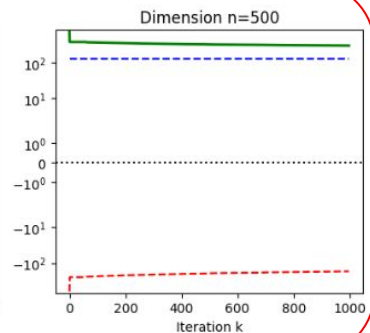
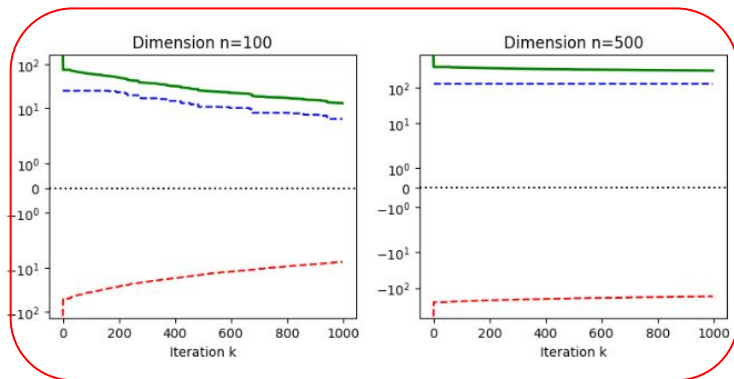
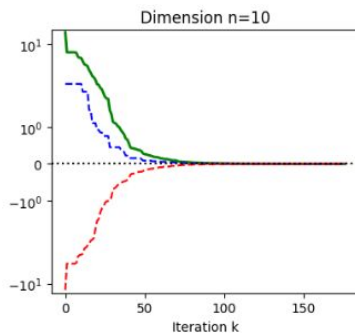
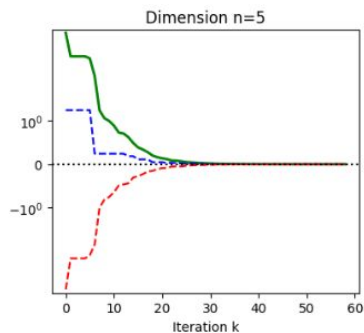
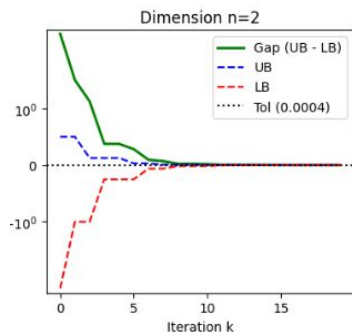
## Projected Subgradient

Converged at iteration: **3,231**  
 Optimal  $x \approx 0.567134$   
 Optimal  $f \approx 0.567149$   
 CPU = 0.206  
Gap = 0.009985

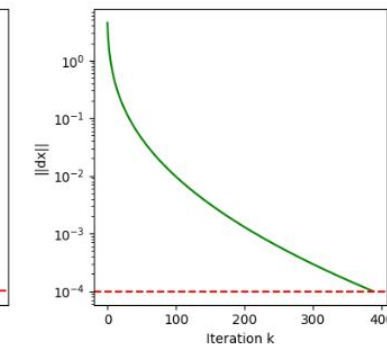
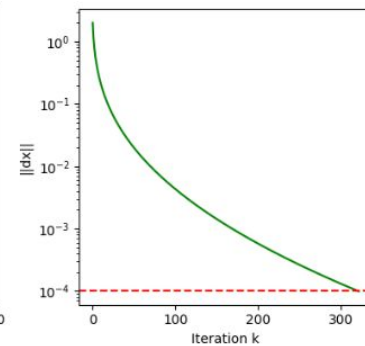
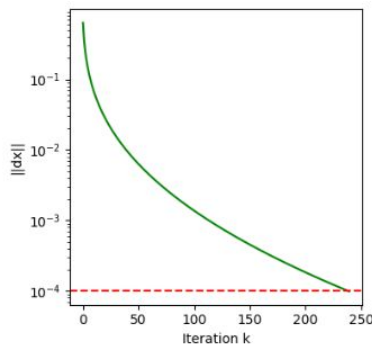
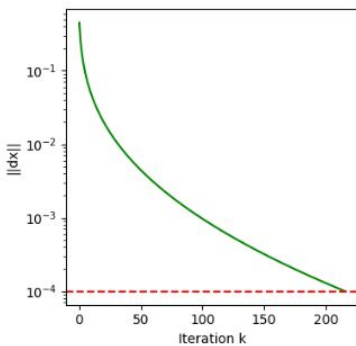
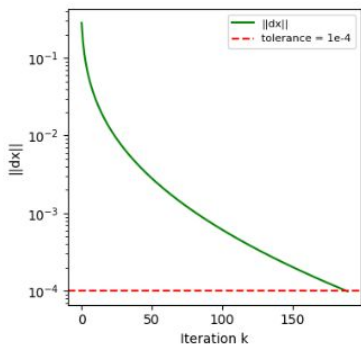


# P4 · Scaling with Dimension n

## Cutting-Plane



## Projected Subgradient



## P4 · Scaling with Dimension $n$

### Cutting-Plane

### Projected Subgradient

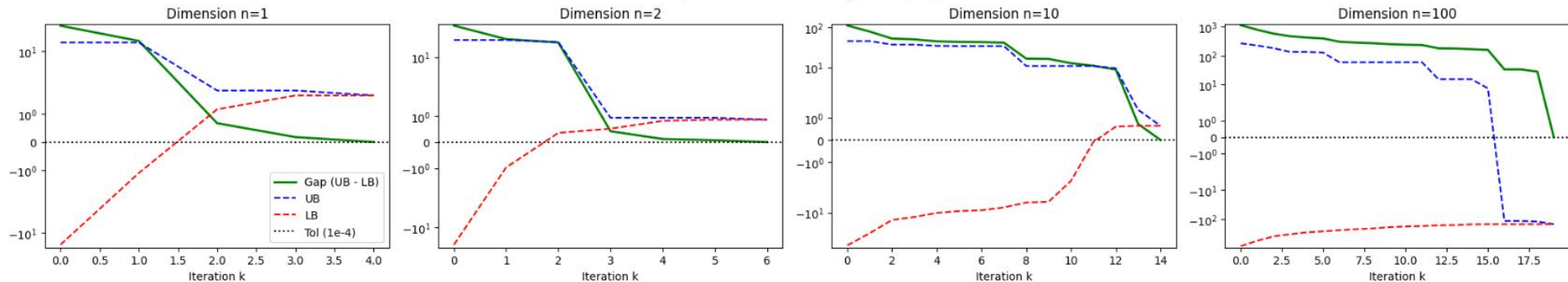
| n   | Iteration Count<br>(Cutting Plane) | CPU Time<br>(Cutting Plane) | Iteration Count<br>(Subgrad) | CPU Time (s)<br>(Subgrad) |
|-----|------------------------------------|-----------------------------|------------------------------|---------------------------|
| 2   | 20                                 | 0.57                        | 190                          | 0.003454                  |
| 5   | 59                                 | 2.21                        | 217                          | 0.003930                  |
| 10  | 177                                | 11.05                       | 240                          | 0.004443                  |
| 100 | 999                                | 563                         | 323                          | 0.005856                  |
| 500 | 999                                | 2857                        | 389                          | 0.008652                  |



# P5 · Scaling with Dimension n (m = 20)

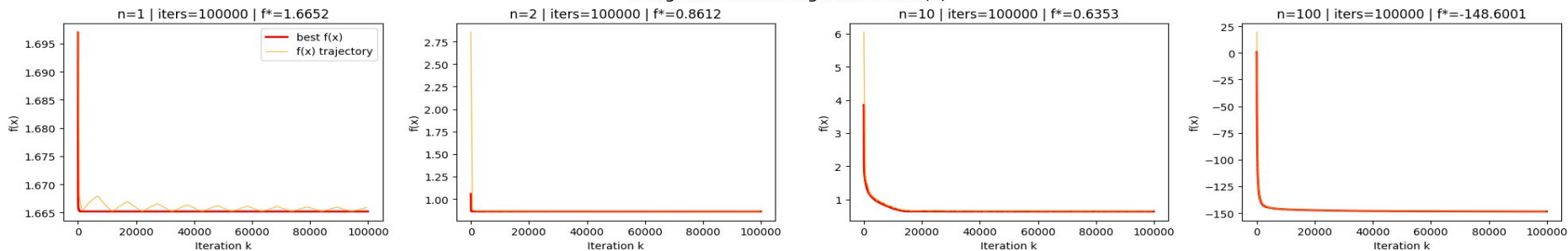
## Cutting-Plane

Kelley's Method Convergence (P5)



## Projected Subgradient

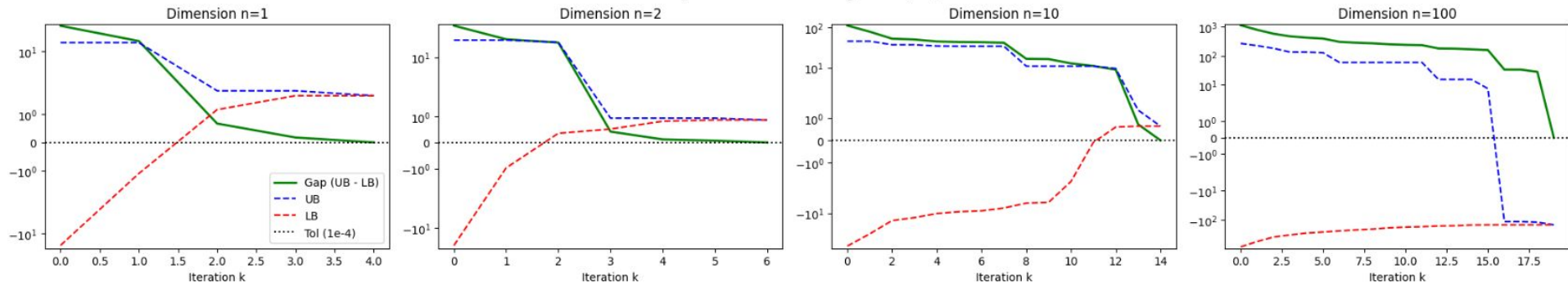
Subgradient Convergence: Best  $f(x)$



# P5 · Scaling with Dimension n (m = 20)

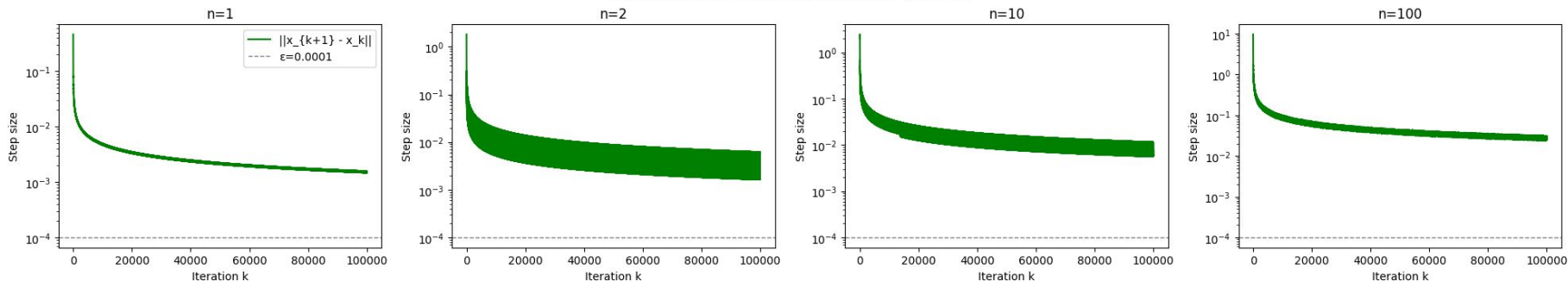
## Cutting-Plane

Kelley's Method Convergence (P5)



## Projected Subgradient

Subgradient Convergence: Step Change



## Cutting-Plane

## Projected Subgradient

| n   | Iteration Count<br>(Cutting Plane) | CPU Time<br>(Cutting Plane) | Iteration Count<br>(Subgrad) | CPU Time (s)<br>(Subgrad) |
|-----|------------------------------------|-----------------------------|------------------------------|---------------------------|
| 1   | 5                                  | 0.062259                    | 100,000                      | 4.112795                  |
| 2   | 7                                  | 0.098097                    | 100,000                      | 1.652612                  |
| 10  | 15                                 | 0.297682                    | 100,000                      | 1.721162                  |
| 100 | 20                                 | 0.696236                    | 100,000                      | 1.742292                  |

### Projected Subgradient

Revised

---

**Algorithm 4** Projected Subgradient Method for Problem 5

---

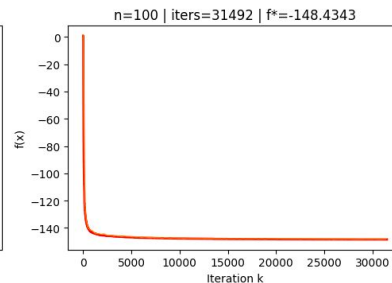
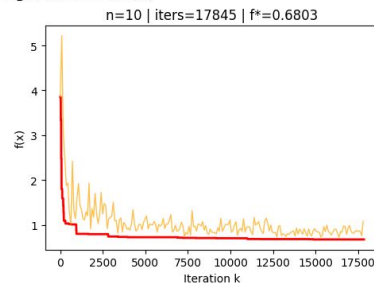
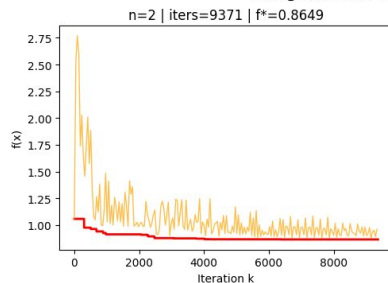
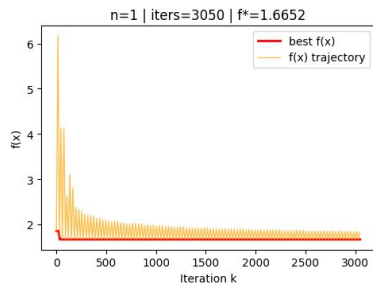
```
1: Initialize:  $x_0 = \mathbf{0}$ ,  $\mathcal{X} = [-10, 10]^n$ ,  $\text{UB} = f(x_0)$ ,  $\varepsilon = 10^{-4}$ ,  
    $K_{\max} = 100000$   
2: for  $k = 1, 2, \dots, K_{\max}$  do  
3:    $g_k \in \partial f(x_{k-1})$   
4:    $\alpha_k = \frac{x_{\text{hi}} - x_{\text{lo}}}{\sqrt{k} \|g_k\|_2}$   
5:    $x_k = \Pi_{\mathcal{X}}(x_{k-1} - \alpha_k g_k)$   
6:    $\text{UB} = \min(\text{UB}, f(x_{k-1}))$   
7:   if  $k > W$  and  $\text{UB}_{k-W} - \text{UB}_k < \varepsilon$  then  
8:     return  $x_k, f(x_k)$   $\triangleright$  no improvement over  $W$  iterations  
9:   end if  
10: end for
```

---

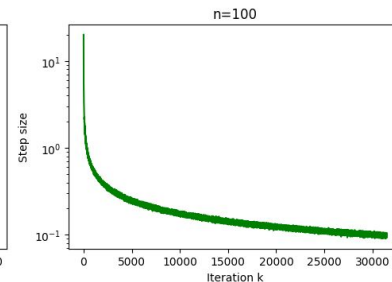
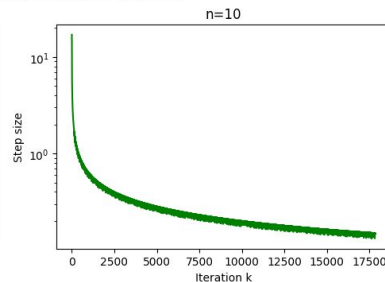
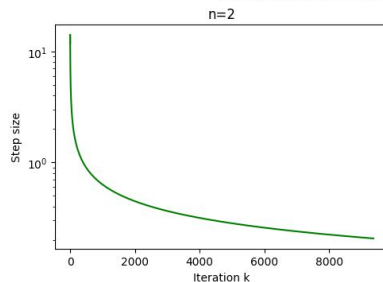
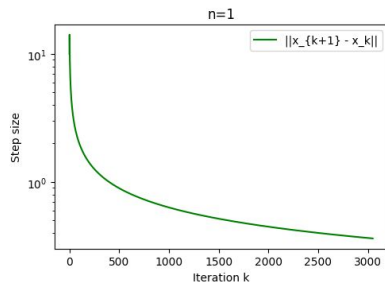
# P5 · Scaling with Dimension n (m = 20)

## Projected Subgradient Revised

Subgradient Convergence: Best  $f(x)$



Subgradient Convergence: Step Change



## Projected Subgradient Revised

## Projected Subgradient Initial

| $n$ | Iteration Count<br>(Cutting Plane) | CPU Time<br>(Cutting Plane) | Iteration Count<br>(Subgrad) | CPU Time (s)<br>(Subgrad) |
|-----|------------------------------------|-----------------------------|------------------------------|---------------------------|
| 1   | 3050                               | 0.088562                    | 100,000                      | 4.112795                  |
| 2   | 9371                               | 0.292785                    | 100,000                      | 1.652612                  |
| 10  | 17845                              | 0.529481                    | 100,000                      | 1.721162                  |
| 100 | 31492                              | 0.994899                    | 100,000                      | 1.742292                  |

## Cutting-Plane

## Projected Subgradient Revised

| $n$ | Iteration Count | CPU Time | $F(x^*)$   | Iteration Count | CPU Time (s) | $F(x^*)$    |
|-----|-----------------|----------|------------|-----------------|--------------|-------------|
| 1   | 5               | 0.062259 | 1.665187   | 3050            | 0.088562     | 1.665197    |
| 2   | 7               | 0.098097 | 0.861193   | 9371            | 0.292785     | 0.864896    |
| 10  | 15              | 0.297682 | 0.634057   | 17845           | 0.529481     | 0.680291    |
| 100 | 20              | 0.696236 | -148.79020 | 31492           | 0.994899     | -148.434274 |

## Cutting-Plane

Projected Subgradient <sup>Revised</sup>

| $n$ | Iteration Count<br>(Cutting Plane) | CPU Time<br>(Cutting Plane) | Iteration Count<br>(Subgrad) | CPU Time (s)<br>(Subgrad) |
|-----|------------------------------------|-----------------------------|------------------------------|---------------------------|
| 1   | 5                                  | 0.062259                    | 3050                         | 0.088562                  |
| 2   | 7                                  | 0.098097                    | 9371                         | 0.292785                  |
| 10  | 15                                 | 0.297682                    | 17845                        | 0.529481                  |
| 100 | 20                                 | 0.696236                    | 31492                        | 0.994899                  |



# Comparison of the two Algorithms Performance

## Cutting-Plane

## Projected Subgradient

|                             |                                                                                       |                                                                                                        |
|-----------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Smooth Functions            | LP is effective but expensive                                                         | The negative of the subgradient is going to point in the direction of the minimum, convergence is fast |
| Kinked Functions            | Each cut is a global underestimator regardless of edge so converges in few iterations | Struggles near the edges, so requires more iterations, but still computationally quick                 |
| Piece-wise linear functions | When the function is a max of affine pieces, converges quickly                        | Slower convergence as it struggles near edges                                                          |
| High dimensional space      | Struggles as $n$ grows given it has $n+1$ variables, adds a constraint per iteration  | Each iteration is quick and grows linearly with $n$                                                    |

# Problem 6: Level Method Algorithm

**Initialization:** Start at  $x_1$ , set Upper Bound ( $UB=\infty$ ), Lower Bound ( $LB_0=-\infty$ ).

## Step 1: Evaluate & Update (same as Kelley's cutting plane method)

- Calculate  $f(x_k)$  and subgradient  $g_k$ .
- Update  $UB_k = \min(UB_{k-1}, f(x_k))$ .

## Step 2: Build the Model

- Add the new tangent plane to memory.
- Construct the piecewise-linear model:  $m_k(x) = \max_i \{f(x_i) + g_i^T(x - x_i)\}$ .

## Step 3: Calculate Gap & Level

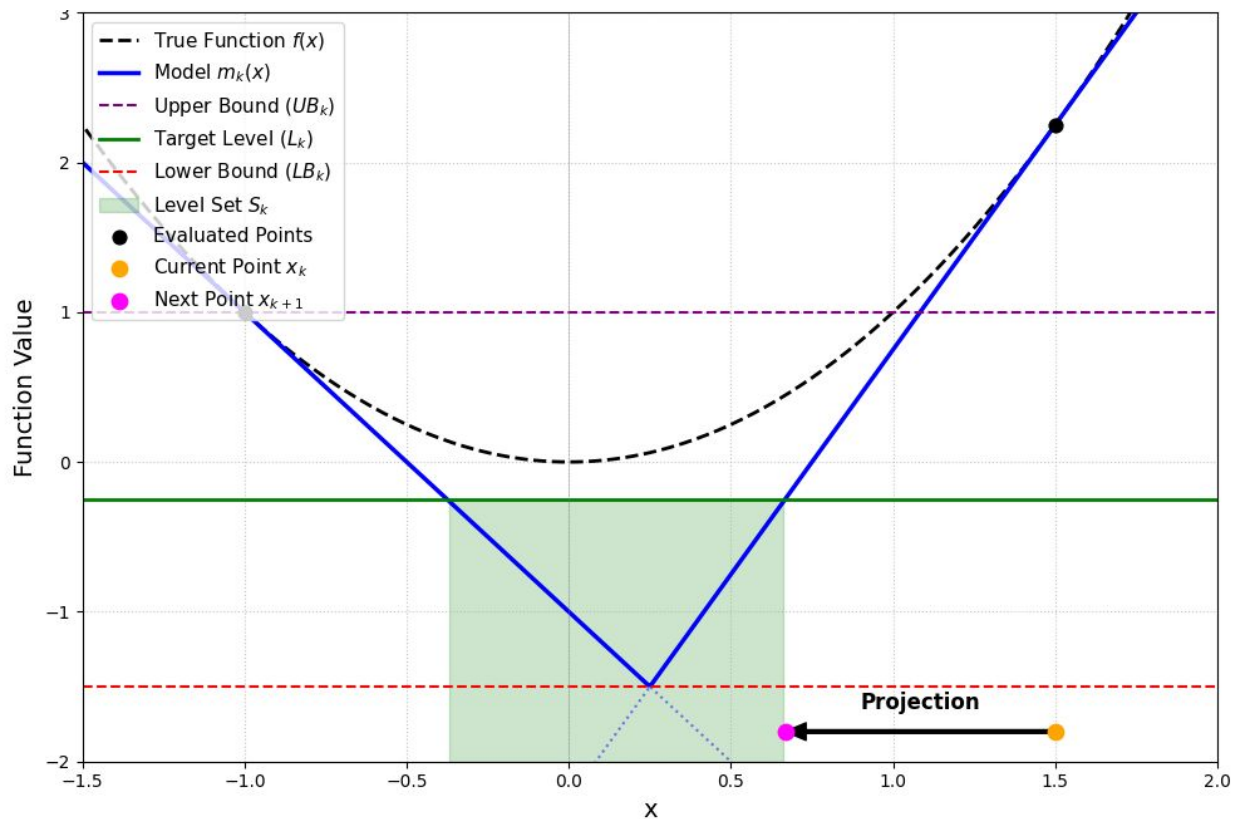
- Find the model's absolute minimum ( $LB_k$ ).
- Check the gap:  $\Delta_k = UB_k - LB_k$ . Stop if  $\Delta_k \leq \epsilon$ .
- Calculate the target level:  $L_k = LB_k + \lambda \Delta_k$  ( $\lambda$  is usually **0.5 or midway**)

## Step 4: Project & Repeat

- Define the Level Set  $S_k$  (the region where  $m_k(x) \leq L_k$ ).
- Project the current point  $x_k$  onto  $S_k$  to find the next point  $x_{k+1}$ .

The level method is similar to the Kelley method in that it builds a piecewise-linear approximation of the objective function using subgradients. It finds the lowest point of the model, and records the best actual function value found at this point. What differentiates it from Kelley's is that instead of jumping straight to the minimum of the model, it draws a target "level"  $L_k$  that at some  $\lambda$  between the bounds. It then restricts the search space to this level set (the region where the model sits below the target level) and projects the *current* point  $x_k$  into this set to find  $x_{k+1}$ .

## Problem 6: Level Method Graphical Representation



In the 2D case it is quicker to use Kelley's but in higher dimension, it is advantageous to not immediately jump to each minimum and avoid the zig-zagging to make smaller steps towards the minimum

# Problem 6: Level Method Description

## Key Strengths:

- Has both the memory of Kelley's Method with the conservative stability of the Projected Subgradient Method
- Eliminates zig-zagging by projecting onto a level set instead of jumping to the absolute minimum, it prevents the violent instability seen in standard cutting-plane methods like Kelley's (avoids getting trapped bouncing between walls)
- You can always guarantee progress because the projection step ensures  $x_{k+1}$  is as close to  $x_k$  as possible while strictly reducing the optimality gap

---

### Algorithm 1 Level Method (Specific to Problem 6)

---

```
1: Initialize:  $x_0 \in [-0.5, 1]^n$ ,  $x_{\text{best}} = x_0$ ,  $\text{UB} = \|x_0\|^2$ ,  $\text{LB} = -\infty$ ,  
    $\varepsilon = 10^{-4}$ ,  $\lambda = 0.5$   
2: for  $k = 1, 2, \dots, K_{\text{max}}$  do  
3:   if  $\|x_{k-1}\|^2 < \text{UB}$  then  
4:      $x_{\text{best}} = x_{k-1}$ ,  $\text{UB} = \|x_{k-1}\|^2$   
5:   end if  
6:   Solve:  $\text{LB} = \min_{x \in [-0.5, 1]^n, \theta} \theta$   
7:           s.t.  $\theta \geq \|x_j\|^2 + 2x_j^\top(x - x_j) \quad \forall j = 0, \dots, k-1$   
8:   if  $\text{UB} - \text{LB} \leq \varepsilon$  then  
9:     return  $x_{\text{best}}, \|x_{\text{best}}\|^2$   
10:  end if  
11:   $L_k = \text{LB} + \lambda(\text{UB} - \text{LB})$   
12:  Solve:  $x_k = \arg \min_{x \in [-0.5, 1]^n} \|x - x_{k-1}\|^2$   
13:           s.t.  $\|x_j\|^2 + 2x_j^\top(x - x_j) \leq L_k \quad \forall j = 0, \dots, k-1$   
14: end for
```

---

## Problem 6: Level Set Experiment - Results #1

### Cutting-Plane (Q4)

### Cutting-Plane + Level Set

| n   | Iteration Count<br>(Cutting Plane) | CPU Time<br>(Cutting Plane) | Iteration Count<br>(Level-Set) | CPU Time (s)<br>(Level-Set) |
|-----|------------------------------------|-----------------------------|--------------------------------|-----------------------------|
| 2   | 20                                 | 0.315404                    | 7                              | 0.233623                    |
| 5   | 59                                 | 1.662454                    | 7                              | 0.244863                    |
| 10  | 177                                | 14.802159                   | 8                              | 0.664217                    |
| 100 | 760                                | 788.220329                  | 10                             | 0.357500                    |
| 500 | 999                                | 3171.120585                 | 10                             | 4.233974                    |

# Problem 6: Level Method Experiment

Hypothesis 1: Manipulating the  $\lambda$  variable to climb towards the upper bound reduces the number of iterations and CPU time required for convergence



Hypothesis 2: The optimal  $\lambda$  value is dependent on the initial distance to the objective's minimum; as the feasible range shifts the starting point further from the optimum, large  $\lambda$  values will outperform small values.



```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import time

def f(x): return np.dot(x, x)
def g(x): return 2 * x

dimensions = [2, 5, 10, 100, 500]
results = []
historysALL = {}
lam = 0.01 # level parameter: 0=stay near UB, 1=push toward LB
tolerance = 0.0001

for n in dimensions:
    print(f"\nRunning n = {n}...")
    x_low = -1.0 * np.ones(n)
    x_high = 0.5 * np.ones(n)

    # — start at corner, far from optimum —————
    x_cur = x_low.copy()
    best_UB = f(x_cur)
    best_LB = -np.inf
    best_x = x_cur.copy()

    cuts = [(f(x_cur), g(x_cur), x_cur.copy())]
    UB_hist, LB_hist, gap_hist = [], [], []

    start = time.time()

    for k in range(1, 1000):
```

## Problem 6: Level Set Experiment - Results #2

Range of  $[-0.5, 1]$

### Larger Level Set

### Smaller Level Set

| n   | $\lambda = 0.5$<br>Iterations | $\lambda = 0.99$<br>Iterations | $\lambda = 0.9$<br>Iterations | $\lambda = 0.1$<br>Iterations | $\lambda = 0.05$<br>Iterations | $\lambda = 0.01$<br>Iterations |
|-----|-------------------------------|--------------------------------|-------------------------------|-------------------------------|--------------------------------|--------------------------------|
| 2   | 7                             | 9                              | 8                             | 8                             | 13                             | 42                             |
| 5   | 7                             | 10                             | 9                             | 9                             | 13                             | 68                             |
| 10  | 8                             | 10                             | 9                             | 9                             | 13                             | 91                             |
| 100 | 9                             | 12                             | 11                            | 10                            | 13                             | 122                            |
| 500 | 10                            | 13                             | 12                            | 10                            | 14                             | 126                            |

## Problem 6: Level Set Experiment - Results #2

Range of  $[-1, 0.5]$

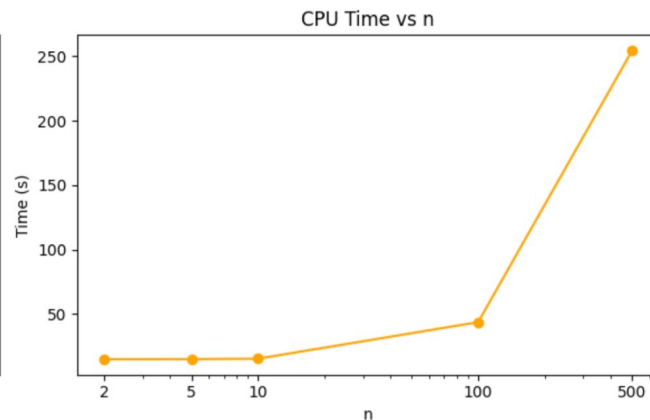
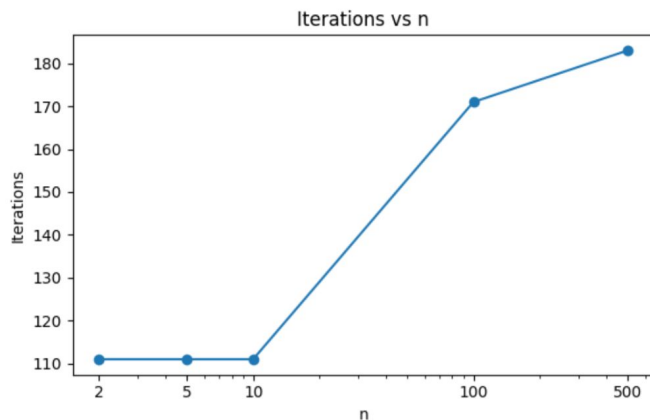
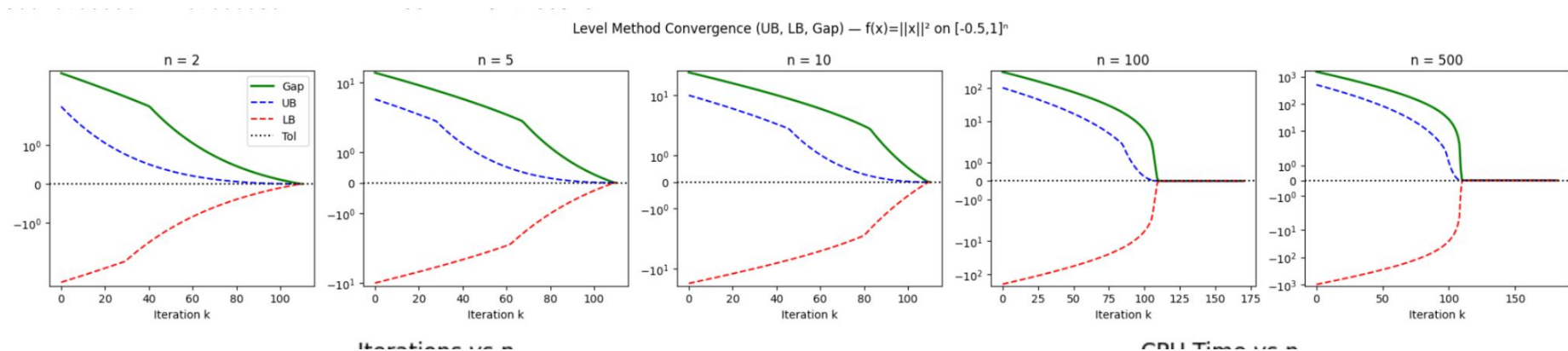
### Larger Level Set

### Smaller Level Set

| n   | $\lambda = 0.5$<br>Iterations | $\lambda = 0.99$<br>Iterations | $\lambda = 0.9$<br>Iterations | $\lambda = 0.1$<br>Iterations | $\lambda = 0.05$<br>Iterations | $\lambda = 0.01$<br>Iterations |
|-----|-------------------------------|--------------------------------|-------------------------------|-------------------------------|--------------------------------|--------------------------------|
| 2   | 7                             | 8                              | 8                             | 23                            | 38                             | 111                            |
| 5   | 7                             | 8                              | 9                             | 23                            | 42                             | 111                            |
| 10  | 8                             | 9                              | 10                            | 24                            | 44                             | 111                            |
| 100 | 10                            | 12                             | 10                            | 24                            | 45                             | 171                            |
| 500 | 10                            | 13                             | 11                            | 29                            | 45                             | 999                            |



# Problem 6: Level Set Experiment - Graphical Pictures



## Problem 6: Level Set Experiment - Conclusion

- ❑ The level set method improves both number of iterations needed for convergence and CPU time, specifically in comparison to the Kelley cutting plane method.
- ❑ Manipulating the  $\lambda$  variable closer to the upper bound (large  $\lambda$ ) increases stability and is changing the aggressiveness of each step
- ❑ By increasing  $\lambda$ , iteration count is reduced by preventing the zig-zagging motion of general cutting plane methods
- ❑ The effectiveness of  $\lambda$  is not independent of the feasible range. If the range shifts and the optimal distance from  $x=0$  is increased, the algorithm requires a larger  $\lambda$  to be efficient.
  - ❑ For the  $[-1,0.5]$  range, the initial cuts are farther from the optimum and thus aggressive  $\lambda$  values fail because they have too small of level sets
- ❑ Generally  $\lambda$  ranges from  $[0.5,0.9]$  provide the most robust performance across various ranges. This allows the algorithm to navigate to the optimum most efficiently even if the starting point is far from the minimum

## Notes for Problem 6

- Discuss impacts of  $\lambda$
- discuss the impacts of the range selected

## Problem 6: N-1 contingency problem

Consider  $m$  generators with cost coefficients  $c_i \sim \mathcal{N}(0, 1)$ , capacity limits  $\bar{p}_i \sim \text{Uniform}(50, 200)$  MW, and total demand  $D = \sum_{i=1}^m \bar{p}_i / 2$ . Define the *N-1 secure economic dispatch* objective:

$$f(p) = \max_{k \in [m]} \left\{ c^\top p + \lambda_k \cdot \max \left( 0, D - \sum_{i \neq k} p_i \right) \right\} \quad (1)$$

Penalty for unmet demand

- (1) Worst case cost if a single generator goes down
- (2) How should we generate to minimize cost from (1)

$$\min_{p \in [0, \bar{p}]^m} f(p) = \min_{p \in [0, \bar{p}]^m} \max_{k \in [m]} \left\{ c^\top p + \lambda_k \cdot \max \left( 0, D - \sum_{i \neq k} p_i \right) \right\} \quad (2)$$

## Problem 6: N-1 contingency problem

- $p \in \mathbb{R}^m$  is the vector of generator output levels, constrained to  $p \in [0, \bar{p}]^m$ , i.e.  $0 \leq p_i \leq \bar{p}_i$  for all  $i \in [m]$ .
- $m \in \{5, 10, 50, 100\}$  is the number of generators, which also equals the number of N-1 contingencies (loss of any single generator).
- $c \in \mathbb{R}^m$  is the vector of operating cost coefficients, with  $c_i \sim \mathcal{N}(0, 1)$  drawn i.i.d.
- $\bar{p} \in \mathbb{R}^m$  is the vector of generator capacity limits, with  $\bar{p}_i \sim \text{Uniform}(50, 200)$  drawn i.i.d.
- $D = \sum_{i=1}^m \bar{p}_i / 2$  is the total system demand, set to half the aggregate capacity.
- $k \in [m]$  indexes the contingency scenario corresponding to the loss of generator  $k$ .
- $\lambda_k \sim \mathcal{N}(0, 1)$  are i.i.d. penalty weights representing the cost of unmet demand under contingency  $k$ .
- $\max\left(0, D - \sum_{i \neq k} \bar{p}_i\right)$  is the unmet demand (load shed) when generator  $k$  is lost and the remaining generators  $\{i \neq k\}$  cannot fully cover  $D$ .